

DOLFIN User Manual

May 25, 2012



Hoffman, Jansson, Logg, Wells

www.fenics.org

Visit <http://www.fenics.org/> for the latest version of this manual.
Send comments and suggestions to dolfin-dev@fenics.org.

Contents

1	Input/output	5
1.1	Files and objects	5
1.2	File formats	7
1.2.1	DOLFIN XML	7
1.2.2	DOLFIN Binary	8
1.2.3	VTK	8
1.2.4	GNU Octave	9
1.2.5	MATLAB	10
1.3	Converting between file formats	10
1.4	A note on new file formats	11
A	DOLFIN Binary file format	15
A.1	General	15
A.1.1	Mesh data	16
A.1.2	Function data	16

A.1.3 Mesh function	16
-------------------------------	----

Chapter 1

Input/output

DOLFIN relies on external programs for pre- and post-processing, which means that computational meshes must be imported from file (pre-processing) and computed solutions must be exported to file and then imported into another program for visualization (post-processing). To simplify this process, **DOLFIN** provides support for easy interaction with files and includes output formats for a number of visualization programs.

1.1 Files and objects

A file in **DOLFIN** is represented by the class `File` and reading/writing data is done using the standard C++ operators `>>` (read) and `<<` (write).

Thus, if `file` is a `File` and `object` is an object of some class that can be written to file, then the object can be written to file as follows:

```
file << object;
```

Similarly, if `object` is an object of a class that can be read from file, then data can be read from file (overwriting any previous data held by the object)

as follows:

```
file >> object;
```

The format (type) of a file is determined by its filename suffix, if not otherwise specified. Thus, the following code creates a `File` for reading/writing data in **DOLFIN** XML format:

```
File file("data.xml");
```

A complete list of file formats and corresponding file name suffixes is given in Table 1.1.

Alternatively, the format of a file may be explicitly defined. One may thus create a file named `data.xml` for reading/writing data in GNU Octave format:

```
File file("data.xml", File::octave);
```

Suffix	Format	Description
.xml/.xml.gz	File::xml	DOLFIN XML
.bin	File::binary	DOLFIN Binary
.pvd	File::vtk	VTK
.m	File::octave	GNU Octave
(.m)	File::matlab	MATLAB

Table 1.1: File formats and corresponding file name suffixes.

Although many of the classes in **DOLFIN** support file input/output, it is not supported by all classes and the support varies with the choice of file format. A summary of supported classes/formats is given in Table 1.2.

► *Developer's note:* Some of the file formats are partly broken after changing the linear algebra backend to PETSc. (Do `grep FIXME` in `src/kernel/io/`.)

Format	Vector	Matrix	Mesh	Function	Sample	Parallel
File::xml	in/out	in/out	in/out	—	—	Partial
File::binary	in/out	—	in/out	in/out	—	Yes
File::vtk	—	—	out	out	—	Yes
File::octave	out	out	out	out	out	No
File::matlab	out	out	out	out	out	No

Table 1.2: Matrix of supported combinations of classes and file formats for input/output in **DOLFIN**.

1.2 File formats

In this section, we give some pointers to each of the file formats supported by **DOLFIN**. For detailed information, we refer to the respective user manual of each format/program.

► *Developer's note:* This section needs to be improved and expanded. Any contributions are welcome.

1.2.1 DOLFIN XML

DOLFIN XML is the native format of **DOLFIN**. As the name says, data is stored in XML ASCII format. This has the advantage of being a robust and human-readable format, and if the files are compressed there is little overhead in terms of file size compared to a binary format.

DOLFIN automatically handles gzipped XML files, as illustrated by the following example which reads a **Mesh** from a compressed **DOLFIN** XML file and saves the mesh to an uncompressed **DOLFIN** XML file:

```
Mesh mesh;

File in("mesh.xml.gz");
in >> mesh;
```

```
File out("mesh.xml");  
out << mesh;
```

The same thing can of course be accomplished by

```
# gunzip -c mesh.xml.gz > mesh.xml
```

on the command-line.

1.2.2 DOLFIN Binary

DOLFIN Binary is the new native HPC format of **DOLFIN** and the only format with support for parallel I/O. As the name says, data is stored in uncompressed binary with some additionally header information. A complete specification of the file format can be found in [appendix A](#)

. As the name says, data is stored in uncompressed binary format.

with support for parallel I/O in the form of MPI I/O [?].

1.2.3 VTK

Data saved in VTK format [7] can be visualized using various packages. The powerful and freely available ParaView [6] is recommended. Alternatively, VTK data can be visualized in MayaVi [2], which is recommended for quality vector PostScript output. Time-dependent data is handled automatically in the VTK format.

The below code illustrates how to export a function in VTK format:

```
Function u;
```



```
File out("data.pvd");  
out << u;
```

The sample code produces the file `data.pvd`, which can be read by ParaView. The file `data.pvd` contains a list of files which contain the results computed by **DOLFIN**. For the above example, these files would be named `dataXXX.vtu`, where `XXX` is a counter which is incremented each time the function is saved. If the function `u` was to be saved three times, the files

```
data000000.vtu  
data000001.vtu  
data000002.vtu
```

would be produced. Individual snapshots can be visualized by opening the desired file with the extension `.vtu` using ParaView.

ParaView can produce on-screen animations. High quality animations in various formats can be produced using a combination of ParaView and MEncoder [3].

► *Developer's note:* Add MEncoder example to create animation.

1.2.4 GNU Octave

GNU Octave [4] is a free clone of MATLAB that can be used to visualize solutions computed in **DOLFIN**, using the commands `pdemesh`, `pdesurf` and `pdeplot`. These commands are normally not part of GNU Octave but are provided by **DOLFIN** in the subdirectory `src/utils/octave/` of the **DOLFIN** source tree. These commands require the external program `ivview` included in the open source distribution of Open Inventor [5]. (Debian users install the package `inventor-clients`.)

To visualize a solution computed with **DOLFIN** and exported in GNU Octave format, first load the solution into GNU Octave by just typing the name of

the file without the `.m` suffix. If the solution has been saved to the file `poisson.m`, then just type

```
octave:1> poisson
```

The solution can now be visualized using the command

```
octave:2> pdesurf(points, cells, u)
```

or to visualize just the mesh, type

```
octave:3> pdesurf(points, edges, cells)
```

1.2.5 MATLAB

Since MATLAB [\[1\]](#) is not free, users are encouraged to use GNU Octave whenever possible. That said, data is visualized in much the same way in MATLAB as in GNU Octave, using the MATLAB commands `pdemesh`, `pdesurf` and `pdeplot`.

1.3 Converting between file formats

DOLFIN supplies a script for easy conversion between different file formats. The script is named `dolfin-convert` and can be found in the directory `src/utils/convert/` of the **DOLFIN** source tree. The only supported file formats are currently the Medit `.mesh` format, the Gmsh `.msh` version 2.0 format and the **DOLFIN** XML (`.xml`) mesh format.

To convert a mesh in Medit `.mesh` format generated by TetGen with the `-g` option, type

```
# dolfin-convert mesh.mesh mesh.xml
```

To convert a mesh in Gmsh `.msh` format type

```
# dolfin-convert mesh.msh mesh.xml
```

In generating a Gmsh mesh, make sure to define a physical surface/volume. It is also possible to convert a mesh from the old **DOLFIN** XML (`.xml`) mesh format to the current one by typing

```
# dolfin-convert -i xml-old old_mesh.xml new_mesh.xml
```

Example meshes can be found in the directory `src/utils/convert/` of the **DOLFIN** source tree.

1.4 A note on new file formats

With some effort, **DOLFIN** can be expanded with new file formats. Any contributions are welcome. If you wish to contribute to **DOLFIN**, then adding a new file format (or improving upon an existing file format) is a good place to start. Take a look at one of the current formats in the subdirectory `src/kernel/io/` of the **DOLFIN** source tree to get a feeling for how to design the file format, or ask at dolfin-dev@fenics.org for directions.

Also consider contributing to the `dolfin-convert` script by adding a conversion routine for your favorite format. The script is written in Python and should be easy to extend with new formats.g

Bibliography

- [1] *MATLAB*, 2005. <http://www.mathworks.com/>.
- [2] *MayaVi*, 2005. <http://mayavi.sourceforge.net/>.
- [3] *MEncoder*, 2005. <http://www.mplayerhq.hu/>.
- [4] *Octave*, 2005. <http://www.octave.org/>.
- [5] *Open Inventor*, 2005. <http://http://oss.sgi.com/projects/inventor/>.
- [6] *ParaView*, 2005. <http://www.paraview.org/>.
- [7] *The Visualization Toolkit (VTK)*, 2005. <http://www.vtk.org/>.

Appendix A

DOLFIN Binary file format

This chapter gives a detailed description of the **DOLFIN** Binary file format.

A.1 General

Each binary file first contains the following generic header:

```
/* Generic header */
typedef struct {
    uint32_t magic;
    uint32_t bendian;
    uint32_t pe_size;
    uint32_t type;
} header;
```

Where, `magic` is the file's binary magic (0xBABE), `bendian` is endian information, always one if the file were written on a big endian system, `pe_size` contains the number of PEs who wrote the data and `type` is the type of data contained in the file: 0 for Mesh data, 1 for Vector data, 2 for Function(s) data and 3 for Mesh function data.

A.1.1 Mesh data

A.1.2 Function data

If the generic headers type is 2, the rest of the file contains one or several functions. Directly following the generic header the file contains a single integer (`sizeof(int)`) which tells how many functions the rest of the file contains.

```
#define FNAME_LENGTH 256

/* Header for function data */
typedef struct {
    uint32_t dim;           /* Geometric dimension of function */
    uint32_t size;          /* Size of functions data (typically dim * nu
double t;                 /* Simulation time */
    char name[FNAME_LENGTH]; /* Name of sample (e.g Velocity, Pressure) */
} fheader;
```

dim: The geometric dimension of the function.

size: Size of the function data (typically `dim * number of vertices`).

t: Simulation time

name: Name of sample (e.g Velocity, Pressure)

A.1.3 Mesh function

Index

File, [5](#)

DOLFIN Binary, [8](#), [15](#)

file formats, [7](#)

I/O, [5](#)

input/output, [5](#)

object, [5](#)

post-processing, [5](#)

pre-processing, [5](#)

XML, [7](#)